

PRECISION

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
CERTIFICATES INSTEAD OF TABLES, AND THE CONSTANTS NOBODY CHECKS

OR I O R
2 0 2 6

CONTENTS

1	THE TWIDDLE CONSTANTS, PROVED AND NOT COPIED	1
1.1	What a twiddle constant is	1
1.2	The attack, and why it works	1
1.3	The same fault, with no attacker	2
1.4	Proth's theorem, which makes the check a proof	2
1.5	The order of the root, exactly, without factoring anything	3
1.6	What it costs on one desktop	3
1.7	Why the size is ours to choose	3
1.8	The group law, and the proof that it is not enough on its own	4
1.8.1	The disproof: the law is necessary and is not sufficient	4
1.8.2	Why the order of the checks matters operationally	5
1.9	On the card, where a second silent wrong answer was waiting	5
1.10	What it costs on the card	6
1.11	What this refuses	6
1.12	Source	6
2	THE RELATION SEARCH, AND WHERE THE CARD STOPS HELPING	7
2.1	What the search returns, and why that matters	7
2.2	The measurement	7
2.3	Which part carries the cost	8
2.4	The consequence, and it is the part worth publishing	8
2.5	What this does not settle	9
3	THE ENGINE WORKBOOK	11
3.1	The schema	11
3.2	Standing constraints	11
	BIBLIOGRAPHY	13

THE TWIDDLE CONSTANTS, PROVED AND NOT COPIED

Purpose: State what it costs to prove a transform’s constants instead of inheriting them, what that proof catches, and why every answer it gives carries the witness establishing it. **Scope:** Proth’s theorem, the order of a root of unity, the group law on a twiddle table, and two silent wrong answers found by running it.

A number theoretic transform is a sum of powers of one constant. Everything the transform does depends on that constant having exactly the order it is supposed to have, and almost nobody checks.

1.1 WHAT A TWIDDLE CONSTANT IS

The transform of a sequence of length n over the integers modulo q is

$$\hat{p}_j = \sum_i p_i \omega^{ij}$$

where ω is an n -th root of unity modulo q . The powers of ω the butterflies evaluate are the **twiddle constants**. In the Cooley-Tukey butterfly they enter as $c = a+b\omega$ and $d = a-b\omega$; in the Gentleman-Sande butterfly as $c = a+b$ and $d = (a-b)\omega$. A transform of length 2^k is k stages of $n/2$ butterflies. The twiddles are touched more often than any other value in the computation and are the only inputs supposed to be constant.

1.2 THE ATTACK, AND WHY IT WORKS

Ravi, Yang, Bhasin, Zhang and Chattopadhyay inject a single electromagnetic fault setting the twiddle constants to zero. The transform does not fail. It returns a value of the ordinary shape carrying far less entropy than it should, and that is enough to recover a Kyber key and to forge a Dilithium signature, including a bypass of Dilithium’s own verification.

The attack works because **nothing downstream can tell a real root of unity from a fiddled one**. The output of a transform with broken twiddles is the same size, the same type and the same range as the output of a correct one. There is no error to raise. A wrong twiddle is not detected, it is used.

1.3 THE SAME FAULT, WITH NO ATTACKER

This work produced that fault condition by carelessness, which is worth recording because it is the cheaper way to arrive at it.

A transform was built on three moduli. The third, 1610612737, was taken on faith because it has the right shape, $3 \cdot 2^{29} + 1$, the form such a modulus must have. Its generator was raised to $(p-1)/L$ to produce a root of unity of order L , exactly as the construction requires, and a check written beside it reported the root was fine.

The check was trial division to one hundred, on a ten digit number. 1610612737 is **composite**, and the proof is one line: $2^{1610612736} \not\equiv 1 \pmod{1610612737}$. Fermat refuses it. The certificate is the witness 2.

Because it is composite, $g^{(p-1)/L}$ had no particular order, the twiddles were wrong, and the transform returned numbers of the ordinary shape that were not the convolution of anything. The two correct moduli in the same set returned zero wrong coefficients out of thirty-two. The composite one returned thirty-two out of thirty-two, and the only reason anybody noticed is that the three were recombined against each other and disagreed.

A copied constant is a constant nobody checked. The moduli general purpose libraries reach for are the same values everywhere, carried between codebases as literals. The tables are correct, and that is not the point. The point is that correctness is being inherited instead of established, and inherited correctness cannot tell you when a value has been fiddled.

1.4 PROTH'S THEOREM, WHICH MAKES THE CHECK A PROOF

Theorem 1.1 (Proth). *Let $N = k \cdot 2^n + 1$ with k odd and $k < 2^n$. Then N is prime if and only if there is an integer a with $a^{(N-1)/2} \equiv -1 \pmod{N}$.*

Every modulus such a transform can use has this shape, because admitting a root of unity of order 2^n is the same as 2^n dividing $N-1$. The theorem therefore reaches every candidate that matters.

Why a witness is a proof. Suppose $a^{(N-1)/2} \equiv -1 \pmod{N}$, and let p be any prime factor of N . Then $a^{(N-1)/2} \equiv -1 \pmod{p}$ as well. The order of a modulo p divides $N-1$ and does not divide $(N-1)/2 = k \cdot 2^{n-1}$. Since $N-1 = k \cdot 2^n$, the order must carry the full 2^n , giving $2^n \mid p-1$ and so $p \equiv 1 \pmod{2^n}$. Every prime factor of N is at least $2^n + 1$.

If N were composite it would have at least two such factors. $N \geq (2^n + 1)^2 > 2^{2n}$. But $k < 2^n$ gives $N = k \cdot 2^n + 1 < 2^{2n} + 1$. The two cannot both hold. N is prime. $\square$

That last step needs the condition $k < 2^n$, and a modulus outside that range is reported as out of range instead of being tested anyway.

Why a witness is easy to find. If N is prime then Euler's criterion makes $a^{(N-1)/2}$ the Legendre symbol of a , which is -1 for every quadratic non-residue, and that is half of all a . A witness is therefore found by trying small integers, and the first one usually works.

Both verdicts are certificates. A witness a with $a^{(N-1)/2} \equiv -1$ proves primality. A witness a with $a^{N-1} \not\equiv 1$ proves compositeness, by Fermat. Neither answer is a probability and neither is a judgement call. The only other outcome is running out of candidates, reported as inconclusive instead of rounded toward either verdict.

1.5 THE ORDER OF THE ROOT, EXACTLY, WITHOUT FACTORING ANYTHING

Proving the modulus prime is not enough. The twiddle is $\omega = g^{(p-1)/L}$, and the step from a generator to a root of unity is where the constant stops being checked.

Proposition 1.2. *For $L = 2^m$, if $\omega^L = 1$ and $\omega^{L/2} \neq 1$, then ω has order exactly L .*

Proof. The order divides $L = 2^m$. It is 2^j for some $j \leq m$. If $j < m$ then 2^j divides $2^{m-1} = L/2$, which would give $\omega^{L/2} = 1$. It does not. $j = m$. $\square$

Two exponentiations settle it completely, with no factoring of anything. This is cheap enough to run on every twiddle before every transform, and it is the step the fault attack removes. It refuses a zeroed twiddle, a root of short order, and a root drawn from a composite modulus, all by the same test.

The generator is proved the same way and just as completely: g generates the whole group if and only if $g^{(p-1)/q} \neq 1$ for every prime q dividing $p-1$. Here $p-1 = k \cdot 2^n$ with k small. Factoring it is factoring k , and the proof is complete instead of partial.

1.6 WHAT IT COSTS ON ONE DESKTOP

One proof is one modular exponentiation, measured on unbounded integers with nothing vectorized and nothing on the card:

bits	decimal digits	one proof
64	19	0.0001 s
128	39	0.0001 s
256	77	0.0002 s
512	154	0.0011 s
1024	308	0.0185 s
2048	617	0.0419 s
4096	1,233	0.2876 s

A 1,233 digit number proved prime in under three tenths of a second, deterministically. Not a probable prime, not a result with a confidence attached. A certificate, with the witness recorded beside it, letting a reader recheck the claim without rerunning anything.

The cost grows as roughly the square of the bit length, since the exponent has N bits and each squaring costs the multiply. This is the same theorem the large prime searches run, where the records are Proth primes of several million digits, and the difference between those runs and this table is only how long one is willing to wait.

1.7 WHY THE SIZE IS OURS TO CHOOSE

The tabled moduli are small because they are sized to a machine word. A lane holds sixty-four bits, a product of two residues has to fit in one, and the modulus therefore stops below 2^{32} . That is a property of the lane and not of the mathematics.

The arithmetic here is on unbounded integers. The proof runs at any width, and moduli far past anything tabled can be produced and proved on demand:

modulus	shape	transform length
18446744069414584321	$(2^{32} - 1) \cdot 2^{32} + 1$	2^{32}
9223372195768565761	$2147483685 \cdot 2^{32} + 1$	2^{32}
39614081257148775820397903873	$140737488355387 \cdot 2^{48} + 1$	2^{48}

The last admits a transform of length 281,474,976,710,656. Nobody tables that, because it does not fit in a lane. **The floor belongs to the format, and the format is a choice.**

1.8 THE GROUP LAW, AND THE PROOF THAT IT IS NOT ENOUGH ON ITS OWN

The twiddle table is not a list of numbers that happen to be useful. It is a group, and saying so precisely licenses the way the transform is implemented.

Proposition 1.3 (The law). *Let ω have order exactly n modulo a prime p , and let $w_j = \omega^j$ for $0 \leq j < n$. Then $\{w_j\}$ is the cyclic group of order n generated by ω , and $w_j w_k = w_{(j+k) \bmod n}$ for every j and k .*

Proof. $w_j w_k = \omega^{j+k}$. Write $j+k = qn + r$ with $0 \leq r < n$. Then $\omega^{j+k} = (\omega^n)^q \omega^r = \omega^r = w_r$, and r is $(j+k) \bmod n$. The identity is $w_0 = 1$ and the inverse of w_j is w_{n-j} . The set is closed, has an identity and has inverses, which makes it a group; it is generated by one element. It is cyclic. $\square$

Two consequences are used directly.

The table sums to zero. For $n > 1$, $\sum_j w_j = (\omega^n - 1)/(\omega - 1) = 0$, exactly, because $\omega \neq 1$ makes the denominator invertible. This is an integer zero and not a small residual.

The table is generated by repeated multiplication. $w_{j+1} = w_j \omega$. Building the whole table costs one modular multiply per entry. In floating point that construction is warned against because error accumulates along the chain. In the exact ring there is no error to accumulate. The cheap construction and the correct one are the same construction.

1.8.1 The disproof: the law is necessary and is not sufficient

Proposition 1.4. *Let ω' have order d where d divides n and $d < n$. Then the table $w'_j = \omega'^j$ satisfies the group law at length n exactly, while being the wrong table.*

Proof. Since d divides n , $\omega'^n = (\omega'^d)^{n/d} = 1$. The argument above then runs unchanged with ω' in place of ω , and $w'_j w'_k = w'_{(j+k) \bmod n}$ holds for every j and k . $\square$

A root of half the required order therefore produces a table that is a genuine homomorphic image of the wrong group. At length 4096 with ω' of order 2048:

invariant	verdict on a root of half the order
$w_0 = 1$	passes
$\sum w_j = 0$	passes
$\sum w_j^2 = 0$	passes
group law on sampled pairs	passes, 0 of 256 failures
order test	fails, the only one that does

Every invariant computable from the table at $O(n)$ cost is blind to it, because each one is a statement the smaller group also satisfies. What separates them is the order, and the order is not a property any single relation among entries exposes. The separation costs two exponentiations, and it catches the case every cheaper test misses.

1.8.2 Why the order of the checks matters operationally

The transform builds its twiddles once by repeated multiplication and reads them at a stride, and the group law permits that. But the group law permits the construction from *any* root whose order divides the length, including a wrong one, and the result would be a transform returning values of the ordinary shape that are not the convolution of anything.

The order proof is therefore not an optional extra beside the optimization. It is the precondition for the optimization being safe, and it has to run first. Stated as a rule: **prove the order, then generate the table**. Reversing those two produces a fast wrong answer.

The change was measured. Replacing an exponentiation per butterfly with one table read cut the transform at length 2^{27} from 2006 ms to 680 ms, a factor of 2.95, and the output digest was identical at every length tested. A correct optimization moves the clock and not the answer, and that check says which one happened.

1.9 ON THE CARD, WHERE A SECOND SILENT WRONG ANSWER WAS WAITING

The device transform runs over three moduli, each carrying its Proth witness beside it in the source:

modulus	shape	witness	order
2013265921	$15 \cdot 2^{27} + 1$	11	2^{27}
2281701377	$17 \cdot 2^{27} + 1$	3	2^{27}
3892314113	$29 \cdot 2^{27} + 1$	3	2^{27}

The first run of that kernel agreed with the host on the first modulus and differed in *every coefficient* on the other two. The property separating them is not primality, which all three have. It is that 2013265921 is below 2^{31} and the other two are above it. In the butterfly, two residues below a modulus above 2^{31} sum past 2^{32} and wrap, silently, in a type that raises nothing. The same is true of the borrow in the difference. Only the smallest modulus never reached the boundary. One arm agreed and two did not.

This is the same fault as the composite modulus, one level down. In both cases an assumption about the arithmetic went unstated, the transform returned values of the ordinary shape,

and nothing in the output said otherwise. The first was a constant nobody proved; the second was a width nobody checked. Neither was found by reading the code. Both were found because the gate ran cases differing in one property. The failure named its own cause.

Widened to 64 bits, all three moduli agree with the host, and the device multiply agrees with the reference at every size tested.

1.10 WHAT IT COSTS ON THE CARD

One full multiply is two forward transforms, a pointwise product and one inverse, per modulus:

transform length	decimal digits carried	one modulus
2^{20}	10.1 million	8.5 ms
2^{22}	40.4 million	49.4 ms
2^{24}	162 million	220 ms
2^{26}	646 million	938 ms
2^{27}	1.29 billion	2006 ms

Three moduli at 2^{27} is about six seconds of device time for a product of a billion decimal digits. The host's own multiply is Karatsuba, measured at 0.930 s for a million digits and 5.610 s for three million, an exponent of 1.635; extrapolated across three orders of magnitude that is some twenty hours for the same product. The extrapolation is stated as an extrapolation, since it reaches well past anything measured directly.

The packing costs nothing, and that part is worth keeping. A large integer is already a run of 32-bit limbs. Handing one to the device and reading one back are the same bytes named twice, and no conversion happens. The device returns the product as three arrays and the caller assembles them at a 32-bit stride. Convolution coefficients run past 2^{32} and have to be carried, and the carrying happens inside that addition, in the host's own big-integer arithmetic. A carry pass written out over a hundred million coefficients would cost more than every transform put together; here there is no pass at all.

1.11 WHAT THIS REFUSES

A modulus outside Proth's range, where k is not below 2^n , since a witness rests on the converse, which does not hold there. A root whose order falls short of the length asked for, the fiddled twiddle exactly. And any verdict without its certificate: every answer carries the witness establishing it.

1.12 SOURCE

Prasanna Ravi, Bolin Yang, Shivam Bhasin, Fan Zhang, Anupam Chattopadhyay. *Fiddling the Twiddle Constants: Fault Injection Analysis of the Number Theoretic Transform*. IACR ePrint 2022/824.

THE RELATION SEARCH, AND WHERE THE CARD STOPS HELPING

Purpose: Measure what an integer relation search costs as a function of the precision it runs at, identify which part of it carries that cost, and state the consequence for whether a device can help. **Scope:** one cell of the search, host arithmetic, with the author's prediction recorded beside the measurement that contradicted it.

Everything below is measured in arithmetic: places of precision, multiplies, operand width, seconds. No figure here counts an operation belonging to any particular subject the search might be pointed at.

2.1 WHAT THE SEARCH RETURNS, AND WHY THAT MATTERS

Almost every measurement in this body of work returns a bound or a null. An integer relation search returns a statement: a closed form either holds or it does not, and anybody can check it in an afternoon without repeating the work that found it.

Its control is the 1996 BBP formula for π . Pointed at base sixteen, power one, stride eight, the pipeline returns an exact relation, and the known formula holds on the same values to 895 bits of 900, the remainder being the series truncation. The gate runs in both directions: it recovers a planted relation exactly and it rejects unrelated numbers.

The relation found is not unique, and an early version of this control demanded that it be. Bailey's compendium notes that the auxiliary sums satisfy zero relations among themselves. The exact relations form a lattice and the known formula is one member of it. Demanding a particular member was the wrong control, and it reported a correct answer as a failure.

2.2 THE MEASUREMENT

One cell, π at base sixteen, power one, stride eight, on the host:

places	seconds	multiplies	mean width	Gram-Schmidt rebuilds
550	1.28	1,048,433	387	1,477
1,100	4.66	1,853,850	730	2,612
2,200	23.32	3,433,155	1,423	4,838
4,400	145.57	6,671,400	2,825	—

Read as growth per doubling of the precision:

doubling	time	exponent	multiplies	mean width
550 to 1,100	3.64×	1.86	1.77×	1.89×
1,100 to 2,200	5.00×	2.32	1.85×	1.95×
2,200 to 4,400	6.24×	2.64	1.94×	1.99×

The exponent is not flat and should not be quoted as though it were. It reads 1.86, then 2.32, then 2.64, rising across the range and still rising at the last step. The figure 2.64 is the largest doubling measured and it is the right one to carry forward, because the smaller cells sit where fixed costs still show. Quoting it as a constant across the whole range would overstate what the four rows support.

2.3 WHICH PART CARRIES THE COST

The measurement was built to test whether the lattice reduction is the culprit, and the answer is that it is not. That is the opposite of what was predicted before the run, and the prediction is recorded here because a chapter reporting only the answer is worth less than one reporting the wrong expectation beside it.

Gram-Schmidt rebuilds grow 1.77× then 1.85× per doubling, which is close to linear in the precision and is the same rate the multiply count grows at over the same two steps. The reduction is not outpacing anything.

The multiply carries it. The mean operand width grows 1.89, 1.95, 1.99 per doubling. It is linear in the precision to within the resolution of four rows. Karatsuba costs width^{1.585}, and a linear width against a roughly linear count of multiplies predicts an exponent of $1 + 1.585 = 2.585$. The measurement returned 2.64.

Agreement to that tolerance on a four-row fit is not a proof of the mechanism, and the mechanism predicts it while the alternative predicts something else. A reduction-bound search would show rebuilds growing faster than the multiply count, and they do not.

2.4 THE CONSEQUENCE, AND IT IS THE PART WORTH PUBLISHING

A device multiply only beats the host above operands of 32,768 bits. That crossover is itself a measured property and it moved once already: 192 ms of context per call meant it originally sat at 16,384 limbs, and one persistent process moved it down to 1,024.

The mean operand in this search is 0.642 times the precision, read directly off the last row, 2825/4400. The precision at which the mean operand reaches the device crossover is therefore

$$32768/0.642 \approx 51,000 \text{ places}$$

Scaling the measured 145.57 s at 4,400 places by $(51000/4400)^{2.64}$ gives about 26 hours for one cell there, and a 240-cell grid is about 261 days.

The precision at which the card becomes the right machine is a precision at which a single cell takes a day. That is the finding. No amount of device is the answer to this search, and that is measured instead of argued: the two thresholds are separated by a factor the hardware does not close, and closing it would need a different multiply instead of a faster one.

2.5 WHAT THIS DOES NOT SETTLE

The exponent is fitted on four rows spanning three doublings, and the largest is the only one near the asymptote. A fifth row would tighten it and has not been run.

The crossover at 32,768 bits is a property of one device and one host, measured on one desktop. It moves with either.

And the mechanism argument is an inference from two rates agreeing with a prediction, not a direct attribution. The reduction is ruled out, since its growth is close to linear and does not outpace the multiply count. What the multiply's exponent is exactly, as against what Karatsuba predicts for it, would need the multiply timed on its own across the same widths.

THE ENGINE WORKBOOK

Purpose: Record every configuration the engine has been run in, what was claimed of it, what would have refuted the claim, and what settled it. **Scope:** the arithmetic, the moduli, the machine, the cost, and the objects the engine has been pointed at.

Status: not formal theory. Everything below is measured and repeatable. None of it has been examined for inefficiency or absurdity, and several entries record the implementation as the thing measured.

3.1 THE SCHEMA

Every entry carries five fields.

field	what it holds
Category	which class the entry belongs to
Hypothesis	one sentence, stated so it can be wrong
Configuration	every parameter needed to re-run it, none implied
Refutation	the result that would have killed it, written before the verdict
Proof	what was run, what came back, which way it settled

Refutation is written before the proof. A hypothesis with no refutation absorbs any result, because nothing was excluded in advance. Every entry below that took long to find was found by a claim stated sharply enough to be caught wrong.

Configuration is complete or the entry is worthless. The engine has twice been wrong about which machine it was running on. Both times a parameter was implied instead of written.

Identifiers are *E-category-number*, assigned once. A refuted entry keeps its number and gains a verdict. Nothing is deleted, since a record of what failed prevents it being retried.

3.2 STANDING CONSTRAINTS

These bound every entry and are not re-argued in each.

The search is precision bound, not time bound. A relation whose coefficients run to D digits cannot be seen below roughly $D \cdot n$ digits of input, because the lattice cannot separate a true relation from an accidental cancellation there. Depth is the only axis with no floor under it.

A device multiply beats the host only above 1,024 limbs, which is 32,768 bits. Below that a device call is mostly pipe. Measured on one host and one card, and it moves with either.

The degree one region is closed. Borwein, Galway and Borwein proved in 2004 that π has no degree one BBP formula in a power of two base beyond the three known. Searching it returns what is already there. It appears below only as a control.

A candidate is not a discovery. Surviving the screen shows only that a relation is not a precision artifact. Most of the searched space is already mapped, and a surviving relation has still to be checked against the compendium.

BIBLIOGRAPHY

Every measurement in this work is built on somebody's published result, and the first purpose of this chapter is that those people are credited by name. A surname in a footnote is not a citation. The second purpose is that the credit is checkable: each source is registered in `orior_citations`, where the copy sits itself, and this bibliography is generated from that registry and pinned to one commit of it. A reference therefore resolves not to a title but to exact bytes.

Credit here does not depend on agreement. Where this work reproduces a published result it says so, and where it fails to reproduce one it says that too, under the same names, because a refutation rests on the original as much as a confirmation does.

REGISTRY COMMIT 2aae1103d28d9b919eae59cc9062aeaffe1fa0b2

DATED 2026-10-01

INVENTORY MANIFEST.tsv, signed; the hashes below are its rows

A reader who disagrees with a number in this research paper can ask which copy it was checked against, and the hash answers exactly. Two copies of a paper are rarely the same bytes.

- [1] J. Borwein, W. Galway, D. Borwein. *Finding and Excluding b -ary Machin-Type Individual Digit Formulae*. Canadian Journal of Mathematics 56(5) 897-925, doi:10.4153/cjm-2004-041-2.
- [2] Prasanna Ravi, Bolin Yang, Shivam Bhasin, Fan Zhang, Anupam Chattopadhyay. *Fiddling the Twiddle Constants: Fault Injection Analysis of the Number Theoretic Transform*. IACR ePrint 2022/824.
Copy held `sources/computer_science/cryptography/eprint_2022_824_ravi_twiddle_constants.pdf`,
SHA-256 `e92ceb7b03f6be366f98f9b41f44986419898255a7eb45f1389f3f310a69189f`.